Comparativa de contenedores On-premise, Contenedores en la nube e Hipervisores.

Instalación de On-Premise

(Alexander Santana Santana)

Primeros pasos

Instalamos el ubuntu server

```
root@alex:/home/alex# sudo apt update && sudo apt upgrade -y
Obj:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:4 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 2 paquetes. Ejecute «apt list --upgradable» para verlos.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Calculando la actualización... Hecho
The following upgrades have been deferred due to phasing:
  libnftables1 nftables
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
```

Instalamos las herramientas necesarias

```
root@alex:/home/alex# sudo apt install -y git curl wget vim net-tools
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
git ya está en su versión más reciente (1:2.43.0-1ubuntu7.3).
curl ya está en su versión más reciente (8.5.0-2ubuntu10.8).
wget ya está en su versión más reciente (1.21.4-1ubuntu4.1).
vim ya está en su versión más reciente (2:9.1.0016-1ubuntu7.9).
net-tools ya está en su versión más reciente (2.10-0.1ubuntu4.4).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
```

Instalamos las herramientas necesarias

```
root@alex:/home/alex# sudo apt install -y git curl wget vim net-tools
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
git ya está en su versión más reciente (1:2.43.0-1ubuntu7.3).
curl ya está en su versión más reciente (8.5.0-2ubuntu10.8).
wget ya está en su versión más reciente (1.21.4-1ubuntu4.1).
vim ya está en su versión más reciente (2:9.1.0016-1ubuntu7.9).
net-tools ya está en su versión más reciente (2.10-0.1ubuntu4.4).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
```

Cambiamos el nombre del host

```
root@alex:/home/alex# sudo hostnamectl set-hostname alexander-server
root@alex:/home/alex# hostnamectl
 Static hostname: alexander-server
       Icon name: computer-vm
         Chassis: vm
      Machine ID: 7d21e3b8921040a9989f1c1f2bac0efe
         Boot ID: 990130823a394c4a8523e21a619a6cbd
  Virtualization: oracle
Operating System: Ubuntu 24.04.4 LTS
          Kernel: Linux 6.8.0-101-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
  Hardware Model: VirtualBox
Firmware Version: VirtualBox
   Firmware Date: Fri 2006-12-01
    Firmware Age: 19y 3month 1w 3d
```

Instalación y configuración del mDNS(Avahi)

```
root@alex:/home/alex# sudo apt install avahi-daemon avahi-utils -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
avahi-daemon ya está en su versión más reciente (0.8-13ubuntu6.1).
avahi-utils ya está en su versión más reciente (0.8-13ubuntu6.1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
root@alex:/home/alex# sudo systemctl status avahi-daemon
● avahi-daemon.service - Avahi mDNS/DNS-SD Stack
     Loaded: loaded (/usr/lib/systemd/system/avahi-daemon.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-03-12 15:18:32 UTC; 9s ago
TriggeredBy: ● avahi-daemon.socket
   Main PID: 3223 (avahi-daemon)
     Status: "avahi-daemon 0.8 starting up."
      Tasks: 2 (limit: 4578)
     Memory: 836.0K (peak: 1.2M)
        CPU: 25ms
     CGroup: /system.slice/avahi-daemon.service
             ├─3223 "avahi-daemon: running [alexander-server.local]"
             └─3224 "avahi-daemon: chroot helper"
```

Configuramos el nombre para las pruebas

```
  GNU nano 7.2                    /etc/avahi/services/alexander-demo.service
<?xml version="1.0" standalone='no'?>
<!DOCTYPE service-group SYSTEM "avahi-service.dtd">
<service-group>
  <name>Alexander Demo Contenedores</name>
  <service>
    <type>_http._tcp</type>
    <port>8080</port>
    <txt-record>path=/</txt-record>
  </service>
</service-group>
```

Reiniciamos y comprobamos

```
root@alex:/home/alex# sudo systemctl restart avahi-daemon
root@alex:/home/alex# avahi-browse -a -t
+ enp0s3 IPv6 Alexander Demo Contenedores                  Web Site             local
+ enp0s3 IPv4 Alexander Demo Contenedores                  Web Site             local
+     lo IPv4 Alexander Demo Contenedores                  Web Site             local
```

Ahora vamos a hacer la estructura para el proyecto(Creamos la carpeta y una subcarpeta)

```
root@alex:/home/alex# mkdir ~/proyecto-alexander-contenedores
root@alex:/home/alex# cd ~/proyecto-alexander-contenedores
root@alex:~/proyecto-alexander-contenedores# mkdir static templates
```

Ahora vamos a crear una aplicación personalizada

```
root@alex:~/proyecto-alexander-contenedores# nano app.py
```

Creamos el Templates principal

```
root@alex:~/proyecto-alexander-contenedores# nano templates/index.html
```

Template de explicación mDNS

```
root@alex:~/proyecto-alexander-contenedores# nano templates/mdns.html
```

Requerimientos

```
root@alex: ~/proyecto-alexar
  GNU nano 7.2                                    requirements.txt
Flask==2.3.2
```

Dockerización

```
root@alex:~/proyecto-alexander-contenedores# nano Dockerfile
```

Construimos la imagen

```
STEP 11/16: RUN pip install --no-cache-dir -r requirements.txt
--> Using cache dae914075541ada95a442e2072ccd84962d629abb20356b7a02ac425db8808af
--> dae914075541
STEP 12/16: COPY app.py .
--> Using cache 0cc4ea4aefbd879e89a46b1d4ce383c8740d55974082db9b6a5258de33d3003b
--> 0cc4ea4aefbd
STEP 13/16: COPY templates/ ./templates/
--> Using cache 0655634519008560dc820fd470ea508c551cf63c58a6ff3b8e491b7665da0f90
--> 065563451900
STEP 14/16: EXPOSE 5000
--> Using cache c0d53e0699375decdce4568a36894398127edb56dd731473430b535dbedb41f8
--> c0d53e069937
STEP 15/16: HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3   CMD curl -f http:/
/localhost:5000/estado || exit 1
--> Using cache c7dde109c04d62a4b0af9d8fa9802cdb878069fc7fb3ae3ea43418df3204750b
--> c7dde109c04d
STEP 16/16: CMD ["python", "app.py"]
--> Using cache df51132bd243c18fa3e2f00455f53429d67c5fbefbbd0b0f52f9237ff861c34a
COMMIT alexander-app:v1
--> df51132bd243
Successfully tagged localhost/alexander-app:v1
df51132bd243c18fa3e2f00455f53429d67c5fbefbbd0b0f52f9237ff861c34a
root@alex:~/proyecto-alexander-contenedores# docker build -t alexander-app:v1 .
```

Verificamos que se haya creado

```
root@alex:~/proyecto-alexander-contenedores# docker images | grep alexander
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
localhost/alexander-app   v1          df51132bd243  About a minute ago  138 MB
```

Scripts de demostración

```
  GNU nano 7.2                                  /root/iniciar-demo.sh *
#!/bin/bash
# Script de Alexander - Inicia la demostración

GREEN='\033[0;32m'
BLUE='\033[0;34m'
NC='\033[0m' # No Color

echo -e "${BLUE}================================${NC}"
echo -e "${GREEN}   PROYECTO DE ALEXANDER${NC}"
echo -e "${BLUE}================================${NC}"

# Matar contenedor anterior si existe
docker stop alexander-santana 2>/dev/null
docker rm alexander-santana 2>/dev/null

# Iniciar nuevo contenedor
echo -e "\n🚀 Iniciando contenedor..."
docker run -d \
  --name alexander-santana \
  -p 8080:5000 \
  --restart unless-stopped \
  alexander-app:v1

# Esperar un momento
sleep 3

# Verificar que funciona
echo -e "\n📊 Estado del contenedor:"
docker ps | grep alexander-santana

echo -e "\n✅ DEMO LISTA!"
echo -e "🌐 Accede desde CUALQUIER ordenador del aula:"
echo -e "${GREEN}   http://alexander-santana.local:8080${NC}"
echo -e "\n📝 O desde el servidor local: curl http://localhost:8080"
```

Verificación

```
  GNU nano 7.2                                  /root/verificar-demo.sh *
#!/bin/bash
# Script de Alexander - Verifica que todo funciona

echo "🔍 VERIFICACIÓN COMPLETA - Alexander"
echo "===================================="

# 1. Verificar mDNS
echo -e "\n1️⃣  Verificando mDNS:"
avahi-browse _http._tcp -t | grep Alexander

# 2. Verificar contenedor
echo -e "\n2️⃣  Verificando contenedor:"
if docker ps | grep -q alexander-santana; then
    echo "   ✅ Contenedor funcionando"
    docker ps | grep alexander-santana
else
    echo "   ❌ Contenedor NO funcionando"
fi

# 3. Probar aplicación localmente
echo -e "\n3️⃣  Probando aplicación:"
if curl -s http://localhost:8080 > /dev/null; then
    echo "   ✅ App responde localmente"
else
    echo "   ❌ App NO responde"
fi

# 4. Mostrar IP actual (por si acaso)
IP=$(hostname -I | awk '{print $1}')
echo -e "\n4️⃣  IP actual (por si necesitas): $IP"

echo -e "\n📌 URL DE ACCESO: http://alexander-santana.local:8080"
```

Los hacemos ejecutabels

```
root@alex:~/proyecto-alexander-contenedores# chmod +x ~/iniciar-demo.sh
root@alex:~/proyecto-alexander-contenedores# chmod +x ~/verificar-demo.sh
```

Demostración paso a paso

Mostramos la ip

```
root@alex:~/proyecto-alexander-contenedores# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:86:c4:19 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.71/24 metric 100 brd 192.168.1.255 scope global dynamic enp0s3
       valid_lft 40844sec preferred_lft 40844sec
    inet6 fe80::a00:27ff:fe86:c419/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 36:88:05:f6:88:79 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
```

verificamos el funcionamiento

```
root@alexander-server:~/proyecto-alexander-contenedores# docker ps | grep alexander-santana
b7f1be6244e7   alexander-app:v1   "python app.py"   4 minutes ago   Up 4 minutes (unhealthy)   0.0.0.0
:8080->5000/tcp, [::]:8080->5000/tcp   alexander-santana
```

Vemos los Logs

```
root@alexander-server:~/proyecto-alexander-contenedores# docker logs alexander-santana
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
 server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [12/Mar/2026 16:04:30] "GET / HTTP/1.1" 200 -
192.168.1.33 - - [12/Mar/2026 16:05:12] "GET / HTTP/1.1" 200 -
192.168.1.33 - - [12/Mar/2026 16:05:12] "GET /favicon.ico HTTP/1.1" 404 -
192.168.1.38 - - [12/Mar/2026 16:06:21] "GET / HTTP/1.1" 200 -
192.168.1.38 - - [12/Mar/2026 16:06:22] "GET /favicon.ico HTTP/1.1" 404 -
192.168.1.33 - - [12/Mar/2026 16:06:47] "GET /estado HTTP/1.1" 200 -
```

Vemos el Puerto

```
root@alexander-server:~/proyecto-alexander-contenedores# docker port alexander-santana
5000/tcp -> 0.0.0.0:8080
5000/tcp -> [::]:8080
```

Probamos el local

```
root@alexander-server:~/proyecto-alexander-contenedores# curl http://localhost:8080 | grep Alexander
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 11040  100 11040    0     0  2742k      0 --:--:-- --:--:-- --:--:-- 3593k
    <title>Alexander Santana Santana - Proyecto Contenedores</title>
            <h1>📦 Proyecto de Alexander Santana Santana</h1>
                    <span class="value full-name">Alexander Santana Santana</span>
                <p>Alumno: <strong>Alexander Santana Santana</strong></p>
                <p><strong>Solución de Alexander Santana Santana:</strong> Acceso por nombre fijo</p>
                    - Alexander Santana Santana
            <p><strong>Alexander Santana Santana</strong> - Proyecto de Contenedores On-Premise © 2026
</p>
```

Comandos para ver las estadisticas, logs e información detallada

```
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I
/O    PIDS
fb90da13c6c1   alexander-santana   0.08%     41.34MiB / 3.806GiB   1.06%     16.4kB / 92.8kB   0B / 25
8kB   4

🚀 Iniciando aplicación de Alexander Santana Santana
   Versión: 1.0
   Puerto: 5000
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
 server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 122-944-056
172.17.0.1 - - [12/Mar/2026 16:38:46] "GET /demo/mdns HTTP/1.1" 200 -
172.17.0.1 - - [12/Mar/2026 16:40:41] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [12/Mar/2026 16:40:41] "GET /estado HTTP/1.1" 200 -
172.17.0.1 - - [12/Mar/2026 16:40:41] "GET /demo/mdns HTTP/1.1" 200 -
192.168.1.33 - - [12/Mar/2026 16:41:37] "GET / HTTP/1.1" 200 -
192.168.1.38 - - [12/Mar/2026 16:42:26] "GET / HTTP/1.1" 200 -
192.168.1.38 - - [12/Mar/2026 16:42:27] "GET / HTTP/1.1" 200 -
192.168.1.38 - - [12/Mar/2026 16:42:27] "GET / HTTP/1.1" 200 -
192.168.1.38 - - [12/Mar/2026 16:42:27] "GET / HTTP/1.1" 200 -
192.168.1.38 - - [12/Mar/2026 16:42:32] "GET /estado HTTP/1.1" 200 -
192.168.1.33 - - [12/Mar/2026 16:45:29] "GET / HTTP/1.1" 200 -
192.168.1.33 - - [12/Mar/2026 16:45:30] "GET /favicon.ico HTTP/1.1" 404 -
root@alexander-server:~/proyecto-alexander-contenedores# docker logs alexander-santana
                        "HostPort": "8080"
                    }
                ]
            },
            "Networks": {
                "bridge": {
                    "IPAMConfig": null,
                    "Links": null,
                    "Aliases": null,
                    "DriverOpts": null,
                    "GwPriority": 0,
                    "NetworkID": "93d48c6ed714dfd42ad489d14b2ae49c9622f412c10fb1515e6020564bb2dbbe",
                    "EndpointID": "c9fd1069846105faadd7421c181d6672c1edcc2d12c0522248cf81a75ed69bcc",
                    "Gateway": "172.17.0.1",
                    "IPAddress": "172.17.0.2",
                    "MacAddress": "fe:26:b6:1f:00:f7",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "DNSNames": null
                }
            }
        }
    }
]
root@alexander-server:~/proyecto-alexander-contenedores# docker inspect alexander-santana
```

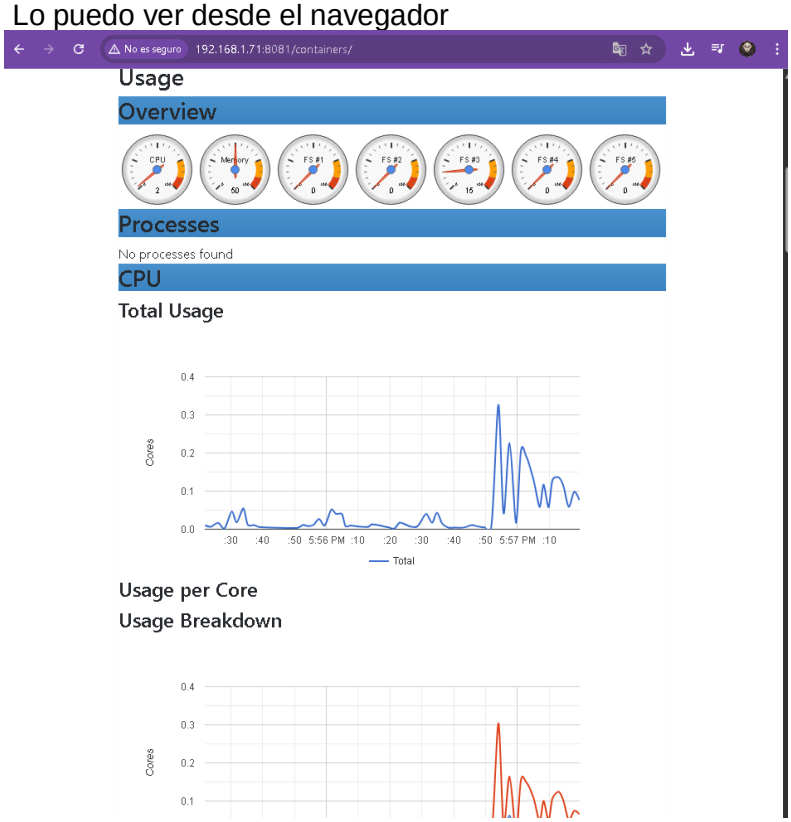
Monitorización visual (cAdvisor)

```
root@alexander-server:~/proyecto-alexander-contenedores# docker run -d \
  --name=cadvisor \
  -p 8081:8080 \
  --restart unless-stopped \
  --volume=/:/rootfs:ro \
  --volume=/var/run:/var/run:rw \
  --volume=/sys:/sys:ro \
  --volume=/var/lib/docker/:/var/lib/docker:ro \
  gcr.io/cadvisor/cadvisor:latest
Unable to find image 'gcr.io/cadvisor/cadvisor:latest' locally
latest: Pulling from cadvisor/cadvisor
2d35ebdb57d9: Pull complete
6bb0a7054e37: Pull complete
14aa4adae9ac: Pull complete
4f4fb700ef54: Pull complete
a43bbcd1f22f: Pull complete
Digest: sha256:3de2bd5203120b866d74a9b283b2ffb8ec382fbf9dc321814700c6ea6f44ec57
Status: Downloaded newer image for gcr.io/cadvisor/cadvisor:latest
e9528a07326314bd87e0a9102abbcc0bafe652a83e2cdd71cffd749bf1e7250e
```

Verifico que está funcionando

```
root@alexander-server:~/proyecto-alexander-contenedores# docker ps | grep cadvisor
e9528a073263   gcr.io/cadvisor/cadvisor:latest   "/usr/bin/cadvisor -…"   About a minute ago   Up 59 s
econds (healthy)   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   cadvisor
```

Lo puedo ver desde el navegador



Mostramos la web que está en el contenedor y que vamos a gestionar.

No es seguro 192.168.1.71:8080

Proyecto de Alexander Santana Santana

Contenedores On-Premise con mDNS

IP No Importa! ¡Solucionado!

Información del Alumno

Nombre completo:	Alexander Santana Santana
Fecha de demostración:	2026-03-12 17:27:13
Curso:	Virtualización y Contenedores
Proyecto:	Comparativa Hipervisores vs Contenedores

MENSAJE PARA EL PROFESOR

Esta es una demostración del proyecto de contenedores on-premise

Alumno: **Alexander Santana Santana**

Profesor, a continuación puede ver:

- Contenedor funcionando con mi aplicación personalizada
- mDNS configurado para acceso por nombre fijo
- Demostración práctica de virtualización con contenedores

SOLUCIÓN mDNS IMPLEMENTADA

Problema: La IP puede cambiar por DHCP

Solución de Alexander Santana Santana: Acceso por nombre fijo

http://alexander-santana.local:8080

- Esta URL funciona SIEMPRE aunque la IP del servidor cambie
- IP actual del servidor: 172.17.0.2 (no es necesario usarla)

Probar mDNS ahora | Ver API Estado

Información del Contenedor

Hostname del servidor:	fa90da13c6c1
ID del contenedor:	fa90da13c6c1
Sistema operativo base:	Linux
Arquitectura:	x86_64
Procesador:	
Tecnología:	Docker + Flask + mDNS (Avahi)

Demostración práctica para el profesor:
Esta es una demostración del proyecto de contenedores on-premise de Alexander Santana Santana para el profesor.

Instrucciones para ver la demostración:

1. **Acceso por nombre fijo (mDNS):** http://alexander-santana.local:8080
2. **Si el nombre no resuelve, usar IP:** http://172.17.0.2:8080
3. **Ver API en JSON:** http://alexander-santana.local:8080/estado
4. **Demostración mDNS:** http://alexander-santana.local:8080/demo/mdns

"Profesor, con esta demostración pretendo mostrar el funcionamiento práctico de contenedores on-premise, la resolución del problema de IPs variables mediante mDNS, y la capacidad de desplegar aplicaciones de forma rápida y eficiente."

```
root@alexander-server:~/monitoring# nano prometheus.yml
root@alexander-server:~/monitoring# cat ~/monitoring/prometheus.yml
global:
  scrape_interval: 15s      # Frecuencia de recolección por defecto
  evaluation_interval: 15s   # Frecuencia de evaluación de reglas

scrape_configs:
  # 1. Prometheus se monitorea a sí mismo
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  # 2. cAdvisor - métricas de TODOS tus contenedores
  - job_name: 'cadvisor'
    scrape_interval: 10s      # Más frecuente para métricas de contenedores
    static_configs:
      - targets: ['cadvisor:8080']   # IMPORTANTE: nombre del servicio, no IP

  # 3. Tu aplicación Flask (si añades endpoint /metrics)
  - job_name: 'alexander-app'
    scrape_interval: 15s
    static_configs:
      - targets: ['192.168.1.71:8080']   # CAMBIA por tu IP real
    metrics_path: /estado      # Usa tu endpoint existente
```

Ejecutamos Prometheus

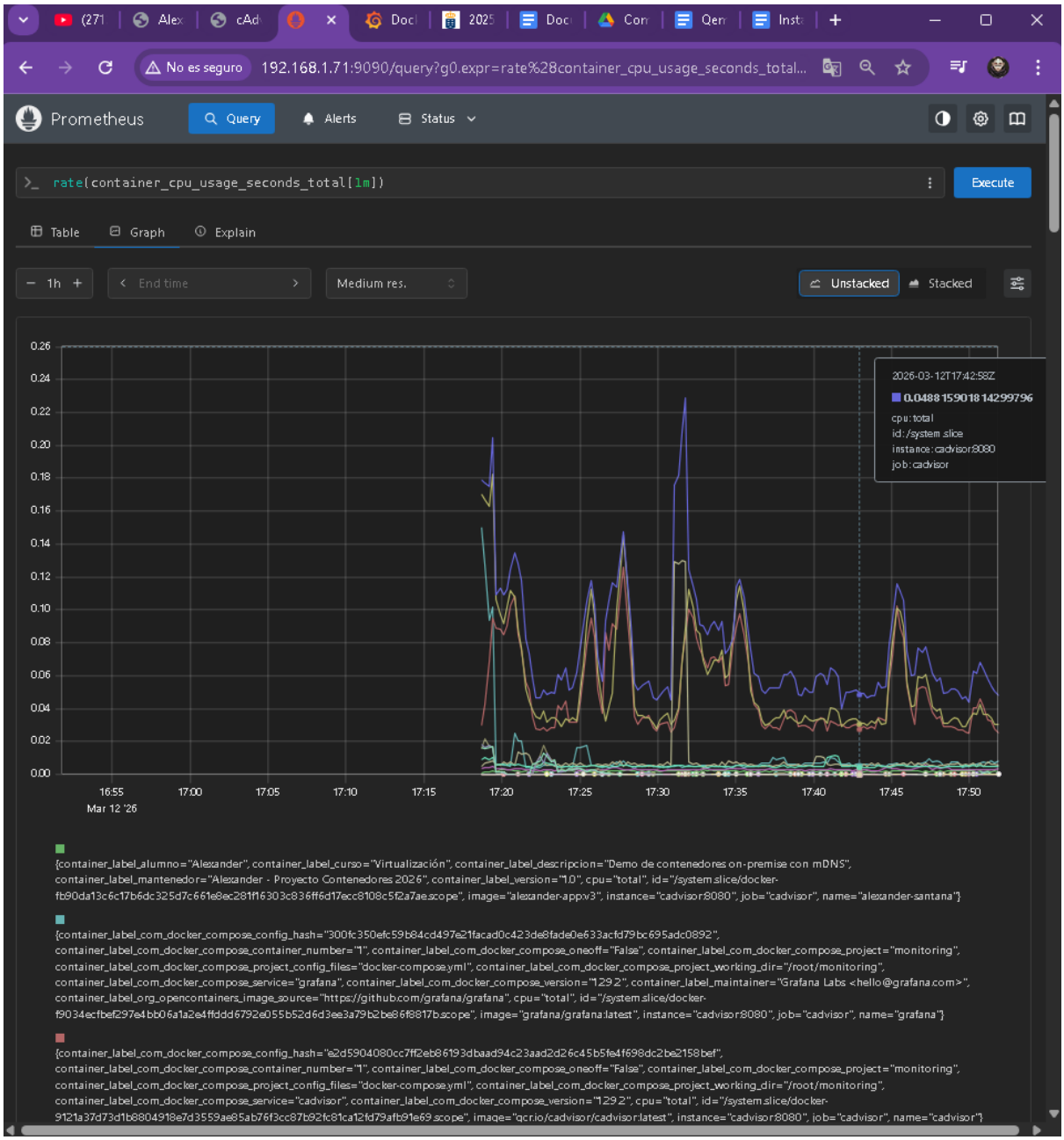
```
root@alexander-server:~/monitoring# docker run -d \
  --name=prometheus \
  -p 9090:9090 \
  -v $(pwd)/prometheus.yml:/etc/prometheus/prometheus.yml \
  prom/prometheus
Unable to find image 'prom/prometheus:latest' locally
latest: Pulling from prom/prometheus
980286587234: Pull complete
2ab9378e6191: Pull complete
95756c9407b7: Pull complete
7accd0adc257: Pull complete
aed72dcc568f: Pull complete
e0997af08d2a: Pull complete
6f79261e6151: Pull complete
6c915ab1ac8a: Pull complete
5b864d736c14: Pull complete
f7e28bb22d10: Pull complete
Digest: sha256:4a61322ac1103a0e3aea2a61ef1718422a48fa046441f299d71e660a3bc71ae9
Status: Downloaded newer image for prom/prometheus:latest
724b27a1bdd0e64ab1b3bb7d38a35638ce8d76b923e1e95fc1460f31227540f6
```

Ejecutamos Grafana

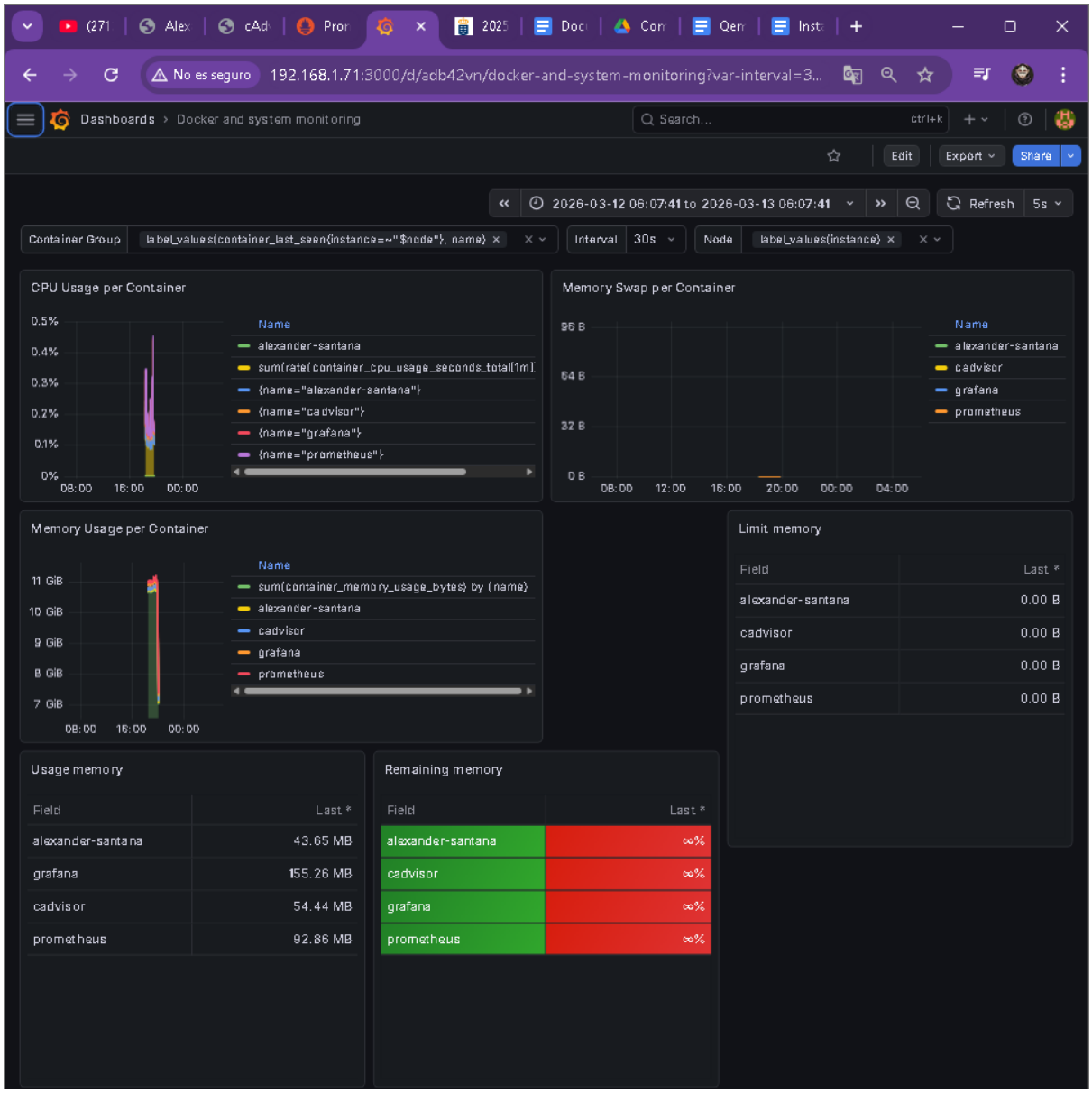
```
root@alexander-server:~/monitoring# docker run -d \
  --name=grafana \
  -p 3000:3000 \
  grafana/grafana
Unable to find image 'grafana/grafana:latest' locally
latest: Pulling from grafana/grafana
589002ba0eae: Pull complete
5f30ee5ce809: Pull complete
92365e648664: Pull complete
01ec7c091b49: Pull complete
e118bc37cc80: Pull complete
0a59c2dec064: Pull complete
e77fbd81b6e3: Pull complete
33ea8387e43a: Pull complete
297d0a340a4a: Pull complete
bf29a0e8c378: Pull complete
4f4fb700ef54: Pull complete
29bc96ad6092: Pull complete
Digest: sha256:e932bd6ed0e026595b08483cd0141e5103e1ab7ff8604839ff899b8dc54cabcb
Status: Downloaded newer image for grafana/grafana:latest
bc6dd9a4d90612c687de10e99d2756b313ecc0d5446b48798ea3fd1ccf5bddd5
```

Ahora las comprobaciones finales.(Las webs de gestión)

Prometheus



Grafana



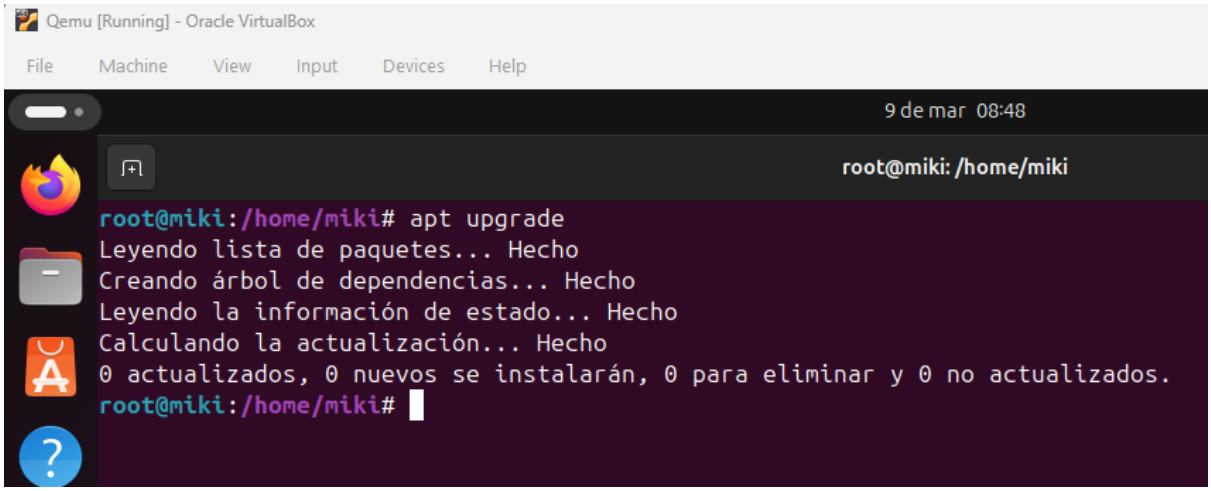
Muestra de carpetas y configuraciones.

```
root@alexander-server:~/monitoring# ls
docker-compose.yml  grafana-datasources  prometheus.yml
root@alexander-server:~/monitoring# ls ~/proyecto-alexander-contenedores/
app.py  diagnostico-completo.sh  Dockerfile  get-docker.sh  requirements.txt  static  templates
root@alexander-server:~/monitoring# ls ~/proyecto-alexander-contenedores/templates/
index.html  mdns.html
```

Hipervisor (Qemu) Instalación

(Miguel Ángel Garrido García)

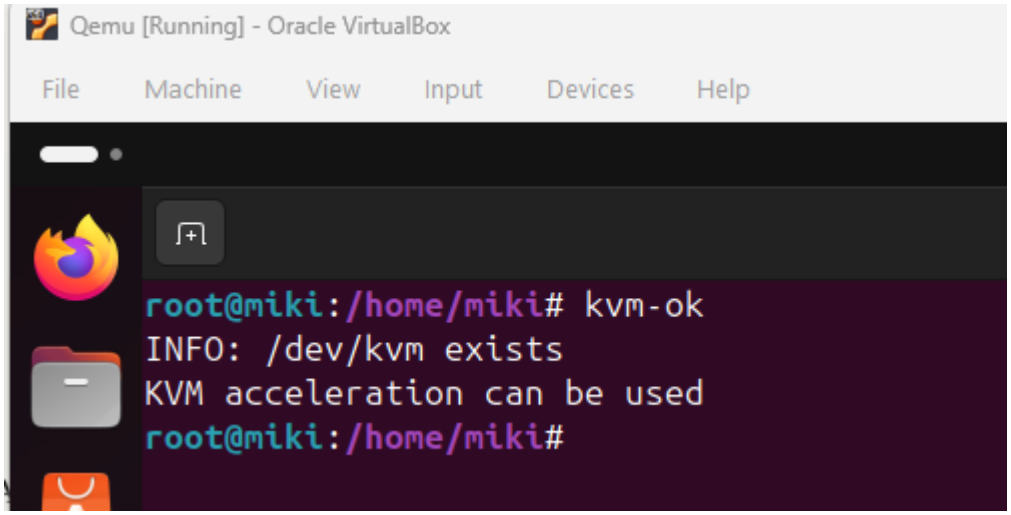
Actualización de la maquina de ubuntu



Instalacion de qemu y sus dependencias

```
root@miki:/home/miki# apt install qemu-kvm qemu-system-x86 libvirt-daemon-system libvirt-clients bridge-utils virt-manag
er
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Nota, seleccionando «qemu-system-x86» en lugar de «qemu-kvm»
qemu-system-x86 ya está en su versión más reciente (1:8.2.2+ds-0ubuntu1.13).
libvirt-daemon-system ya está en su versión más reciente (10.0.0-2ubuntu8.12).
libvirt-clients ya está en su versión más reciente (10.0.0-2ubuntu8.12).
bridge-utils ya está en su versión más reciente (1.7.1-1ubuntu2).
virt-manager ya está en su versión más reciente (1:4.1.0-3ubuntu0.1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
root@miki:/home/miki#
```

Comporbaremos el kvm



En este caso no se podia por la cpu por que es una maquina virtual por eso lo engañaremos este es uno de los puntos negaritvos y la forma de arreglarlo esta abajo

Windows no encuentra `VBoxManage` porque no está en el PATH. Usa la ruta completa:

powershell
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" list vms

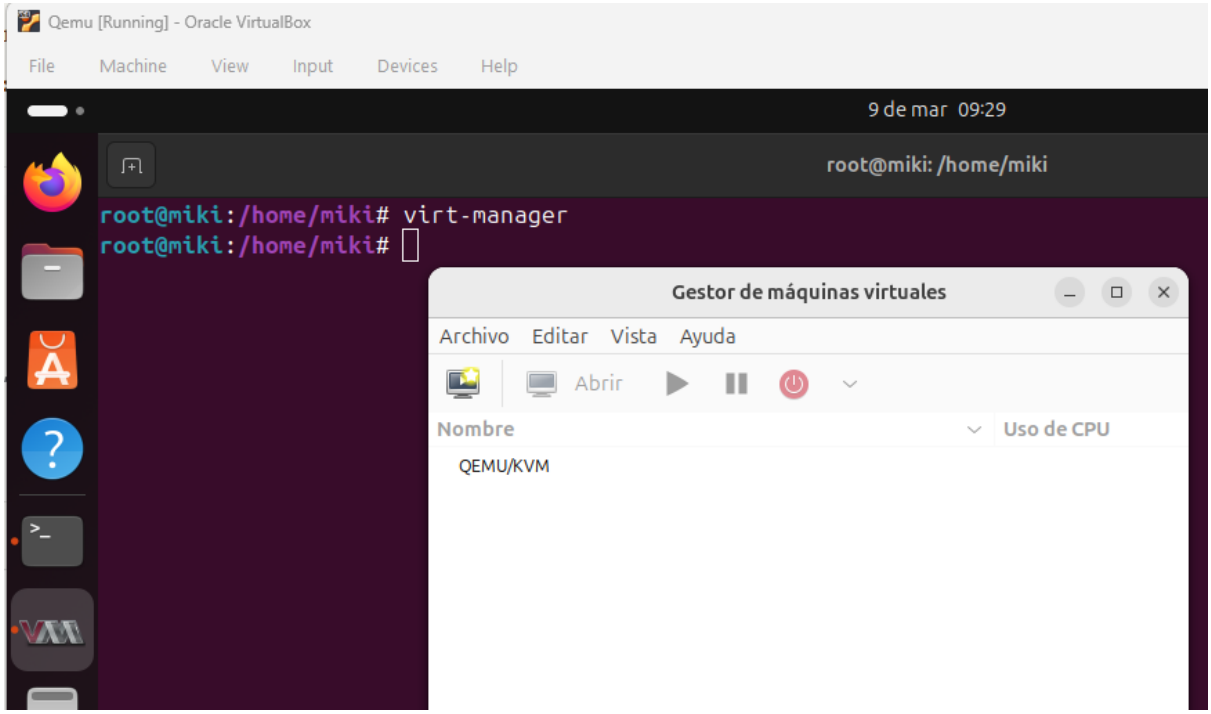
Y luego:

powershell
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" modifyvm "Qemu" --nested-hw-virt on

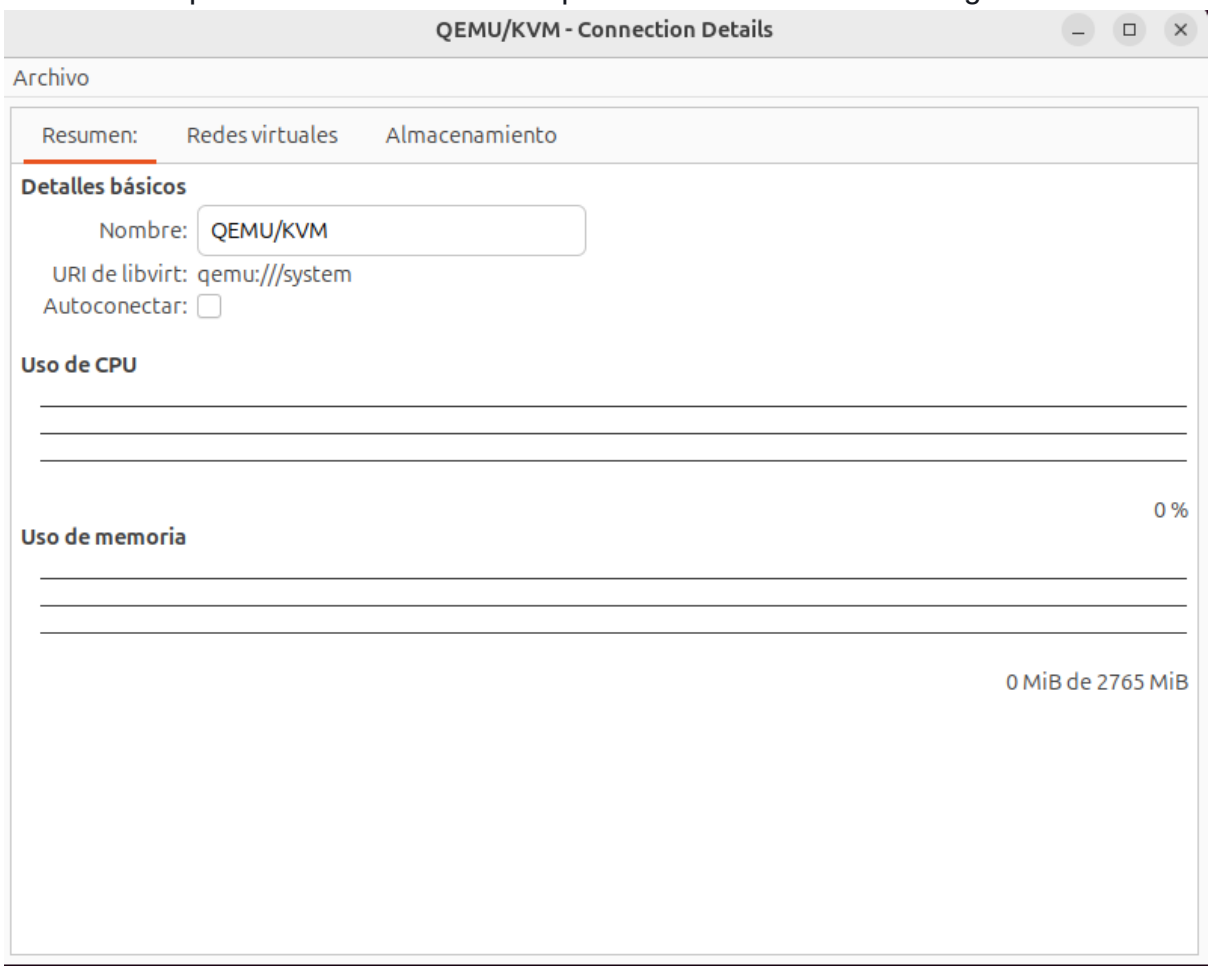
Continuamos con la creacion de los usuario

```
root@miki:/home/miki# usermod -aG libvirt $(whoami)
root@miki:/home/miki# usermod -aG kvm $(whoami)
root@miki:/home/miki# newgrp libvirt
root@miki:/home/miki#
```

Mediante el vm podremos gestionar las maquinas virtuales



mediante el qemu utiliza el sistema vm para visualizarlo de manera grafica



lo malo de qemu es que las maquinas que quieras monitorear y guardar la informacion ya que qemu controla especificamente eso es que tendria que ser instalarlas en la virt-install

Hata aqui es lo que a base debido a errores y contratiempos en estas semanas tengo pensado terminar con el hipervisor con las maquinas y ver algún otro modo de visualizar el tema de máquinas aparte del vm

Contenedores en la nube (AWS)

(Abián García Martel)

¿Por qué hacerlo con una instancia y podman, y no por el propio programa de contenedores de AWS llamado AWS ECS/Fargate?

Esto es debido a que este método a parte de tener un mayor control sobre el entorno del propio contenedor también nos evitamos tener que aumentar nuestro plan de aws y por tanto pagamos menos, para evitar esto he preferido hacerlo por este método el cual es mas barato pero un pelin mas complejo sin embargo esa complejidad nos confiere un mayor control.

antes de abrir la instancia habilitamos el puerto 8080 tanto en entrada como salida:

Filtrar reglas

Nombre	ID de la regla del grupo d...	Intervalo de pu...	Protocolo	Origen	Grupos de seguridad	Descripción
–	sgr-0251f2ea0277c4e82	22	TCP	0.0.0.0/0	launch-wizard-11	–
–	sgr-07d463e7abf5a80aa	80	TCP	0.0.0.0/0	launch-wizard-11	–
–	sgr-039835d4c6e4b8313	443	TCP	0.0.0.0/0	launch-wizard-11	–
–	sgr-0eb8f621046296fb8	8080	TCP	0.0.0.0/0	launch-wizard-11	–

▼ Reglas de salida

Filtrar reglas

Nombre	ID de la regla del grupo d...	Intervalo de pu...	Protocolo	Destino	Grupos de seguridad	Descripción
–	sgr-026d24be3db58789c	8080	TCP	0.0.0.0/0	launch-wizard-11	–
–	sgr-0d15a877857ae95f3	Todo	Todo	0.0.0.0/0	launch-wizard-11	–

miramos la carpeta de la página :

```
root@ip-172-31-29-3:/home/ubuntu/proyecto_5# ls
Dockerfile  css  errores  html  imagenes  index.html  scripts  secciones
root@ip-172-31-29-3:/home/ubuntu/proyecto_5#
```

creó un dockerfile en este caso es sencillito por que se trata solo de una página web de apache:

```
  GNU nano 7.2                                                    Dockerfile
FROM docker.io/library/httpd:2.4

# Copamos todo el contenido del proyecto al directorio web de Apache
COPY . /usr/local/apache2/htdocs/

# Opcional: si quieres activar mod_rewrite (no suele hacer falta para sitios estáticos)
# RUN sed -i '/LoadModule rewrite_module/s/^#//g' /usr/local/apache2/conf/httpd.conf
```

construyó el contenedor con la carpeta de la página web dentro:

```
root@ip-172-31-29-3:/home/ubuntu/proyecto_5# podman build -t proyecto5 .
```

compruebo que se creó el contenedor

```
root@ip-172-31-29-3:/home/ubuntu/proyecto_5# podman ps -a
CONTAINER ID  IMAGE                      COMMAND            CREATED      STATUS                  PORTS                   NAMES
3493f3045e94  localhost/proyecto_5:latest  httpd-foreground  2 weeks ago  Exited (0) 2 weeks ago  0.0.0.0:8080->80/tcp  proyecto_5_container
```

ejecutó el contenedor:

```
root@ip-172-31-29-3:/home/ubuntu/proyecto_5# podman start proyecto_5_container
proyecto_5_container
```

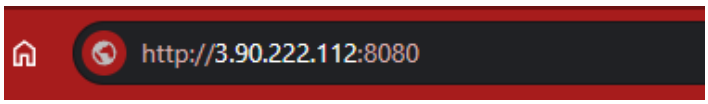
compruebo que está en ejecución:

```
root@ip-172-31-29-3:/home/ubuntu/proyecto_5# podman ps
CONTAINER ID  IMAGE                      COMMAND            CREATED      STATUS          PORTS                   NAMES
3493f3045e94  localhost/proyecto_5:latest  httpd-foreground  2 weeks ago  Up 10 seconds   0.0.0.0:8080->80/tcp  proyecto_5_container
```

Dirección IPv4 pública

3.90.222.112 | dirección abierta

introducimos la siguiente url ya que la ip es la pública de la instancia:



y vemos la página (por si tienes curiosidad es una pagina que cree hace mucho tiempo es muy funcional a nivel de js y la use mayormente para probar trucos de diseño gráfico):

ya para terminar y como prueba de que es el contenedor lo que se ejecuta y no la pagina dentro de apache como tal hacemos este comando:

```
<script src="scripts/js/funciones.js"></script>
</body>root@ip-172-31-29-3:/home/ubuntu/proyesudo ss -tulpn | grep 8080ep 8080
tcp   LISTEN 0      4096           0.0.0.0:8080      0.0.0.0:*    users:(("conmon",pid=1647,fd=5))
root@ip-172-31-29-3:/home/ubuntu/proyecto_5#
```

también comprobamos que el directorio de páginas de apache solo tiene la por defecto:

```
root@ip-172-31-29-3:/home/ubuntu/proyecto_5/html# cd /var/
root@ip-172-31-29-3:/var# ls
backups  cache  crash  lib  local  lock  log  mail  opt  run  snap  spool  tmp  www
root@ip-172-31-29-3:/var# cd www
root@ip-172-31-29-3:/var/www# ls
html
root@ip-172-31-29-3:/var/www# cd html/
root@ip-172-31-29-3:/var/www/html# ls
index.html
root@ip-172-31-29-3:/var/www/html# cat index.html
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
  * {
    margin: 0px 0px 0px 0px;
    padding: 0px 0px 0px 0px;
  }

  body, html {
```

Principales ventajas de contenedores en la nube.

Cuando comparas todo eso con usar contenedores gestionados en la nube, la diferencia es evidente. La nube elimina por completo la necesidad de gestionar servidores, sistemas operativos, redes internas o fallos de hardware: tú solo subes la imagen y el servicio se encarga del resto. Los recursos se ajustan automáticamente según la carga, lo que evita pagar por máquinas sobredimensionadas o quedarte corto en momentos de pico. Además, la integración con balanceadores, seguridad, monitorización y redes privadas viene de serie, lo que reduce muchísimo el tiempo de despliegue y el esfuerzo de mantenimiento. En conjunto, la nube convierte un proceso manual y lleno de pasos en algo más ágil, más flexible y más eficiente, especialmente cuando el proyecto crece o necesita disponibilidad continua.